

Minimax Policy Analysis

1. Introduction

Deterministic games serve as foundational environments for evaluating artificial intelligence and decision-theory strategies. In such settings, complete state observability enables tree-search algorithms to guarantee optimal decision making. In this report, we explore four distinct abstract strategy games—Halving Game, Connect Four, Tic-Tac-Toe, and Nim—and analyze the reason that make them suitable for minimax algorithm.

- **Halving Game**

- ▶ Starts with a finite pile of n stones, where n is randomly initialized between 15 and 50.
- ▶ Two players alternate turns taking actions: either subtract exactly 1 stone, or halve the remaining pile ($\lfloor \frac{n}{2} \rfloor$).
- ▶ The player who takes the last stone (bringing n to 0) wins the game.
- ▶ **Minimax Suitability:** The action space per node is strictly bounded ($|A(s)| \leq 2$), and every move strictly decreases n , driving the game state monotonically toward termination. This guarantees a bounded state space without infinite loops, making complete state-space exploration viable.

- **Connect Four**

- ▶ Played on a bounded grid (4×4 , 5×5 , or 6×7). Players alternate dropping tokens into columns, which fall to the lowest available row due to gravity.
- ▶ The objective is to align 4 identical tokens horizontally, vertically, or diagonally.
- ▶ **Minimax Suitability:** It is a deterministic game with zero randomness once initialized. Every token placement changes the grid in a fully predictable manner, allowing minimax to predict downstream state transitions accurately.

- **Tic-Tac-Toe**

- ▶ Played on a 3×3 grid where two players alternate placing 'X' and 'O' tokens into empty cells.
- ▶ Winning requires aligning 3 matching symbols along a row, column, or diagonal.
- ▶ **Minimax Suitability:** With an exceptionally small search space (9! states maximum), complete minimax tree evaluation guarantees an unbeatably optimal policy (draw or win).

- **Nim**

- ▶ Played with multiple distinct heaps of discrete counters or objects.
- ▶ On each turn, a player selects a single heap and removes any number of objects n ($n \geq 1$), provided all objects are taken from that same heap.
- ▶ Under the standard normal-play convention, the player who takes the last remaining object from the final non-empty heap wins the game.
- ▶ **Minimax Suitability:** Nim features a finite, completely observable state space with zero randomness. Every move strictly reduces the total count of objects across all heaps, ensuring a state progression that guarantees deterministic termination and straightforward utility back-propagation.

2. Strategy and Implementation

Our focus was placed on the formal implementation and empirical evaluation of the Halving Game engine. The codebase structures the environment around explicit state representations, strict transition operations, and a unified recursive minimax decision agent.

Game State Representation & Engine Operations

A game state s is formally represented as a state tuple:

$$s = (n, p)$$

where $n \in \mathbb{N}_0$ denotes the number of stones remaining in the pile, and $p \in \{\text{False}, \text{True}\}$ is a boolean value indicating whose turn it is (False for Player 1, True for Player 2).

The core operational logic of the engine includes:

- **start_state()**: Generates a random initial stone count $n_0 \in [15, 50]$ and sets the initial player boolean to `False` (Player 1).
- **actions(s)**: Identifies legal moves. If $n = 0$, it returns an empty list $\emptyset$ (terminal). Otherwise, it returns `[True, False]`, where `True` represents subtracting 1 stone and `False` represents removing half ($\lfloor \frac{n}{2} \rfloor$).
- **next(s, action)**: Computes the successor state $s' = (n', !p)$ by applying the chosen action:

$$n' = \begin{cases} n - 1 & \text{if action} = \text{True} \\ \lfloor \frac{n}{2} \rfloor & \text{if action} = \text{False} \end{cases}$$

- **is_end(s)**: Evaluates to `True` if $n = 0$, satisfying the terminal condition.
- **utility(s)**: Evaluates payoff at terminal state ($n = 0$):

$$U(s) = \begin{cases} +1 & \text{if player} = \text{True} \quad (\text{Player 1 took final stone}) \\ -1 & \text{if player} = \text{False} \quad (\text{Player 2 took final stone}) \end{cases}$$

- **successors(s)**: Iterates through all available actions from s to generate and return a list of all immediate successor states.

Recursive Minimax Algorithm & Enhancements

The agent performs a recursive depth-first traversal of the game tree. Because every move monotonically reduces n , terminal states are guaranteed within at most n_0 steps. The core recursive search evaluates utility values as follows:

$$\text{Minimax}(s) = \begin{cases} U(s) & \text{if } \text{is_end}(s) \\ \max_{a \in A(s)} \text{Minimax}(\text{next}(s, a)) & \text{if } p = \text{False} \quad (\text{Maximizing Player 1}) \\ \min_{a \in A(s)} \text{Minimax}(\text{next}(s, a)) & \text{if } p = \text{True} \quad (\text{Minimizing Player 2}) \end{cases}$$

- **Model Enhancement**: Rather than instantiating separate agent classes for the Maximizing and Minimizing players, our `MinimaxAgent` dynamically inspects `game.player(state)`. It maximizes utility when $p = \text{False}$ and minimizes when $p = \text{True}$. This single unified model performs optimally regardless of whether it plays as Player 1 or Player 2.

3. Simulation Design

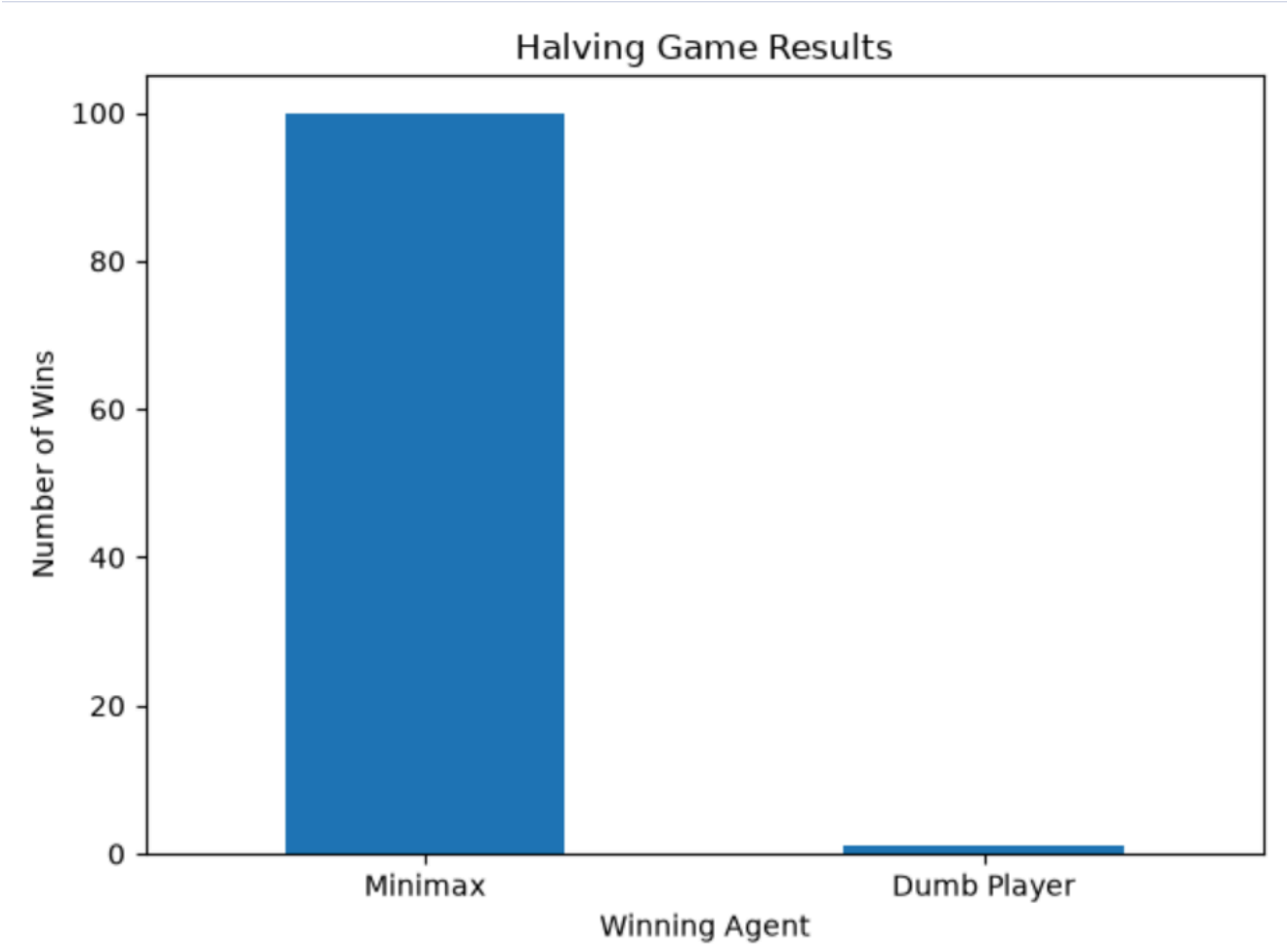
To empirically test our model, an automated simulation suite was designed to pit the **Minimax Agent** against a baseline **Dumb Agent** across 100 matches.

- **Agent Competitors**:
 - ▶ **Minimax Agent**: Executes optimal complete search via `minimax_action()` to choose moves guaranteeing highest achievable terminal utility.
 - ▶ **Dumb Agent**: A deterministic baseline policy that always selects action `True` (subtracting exactly 1 stone on every move).
 - **Fairness Controls**: Role assignments (Player 1 vs Player 2) were randomized evenly (50% probability per game) to eliminate first-mover bias.
 - **Data Collection & Storage**: For each match, the simulation logged key statistical metrics and exported them directly to `halving_results.csv` using Python:
 1. Game Iteration Index
 2. Initial Stone Count (n_0)
 3. Minimax Player Assignment (`True/False`)
 4. Winning Agent Identity (`Minimax` or `Dumb`)
 5. Total Game Length (Moves Elapsed)
-

4. Results Discussion

Across 100 automated simulation games, the **Minimax Agent** demonstrated complete domain dominance over the naive heuristic policy.

Performance Metric	Simulation Value
Total Simulated Games	100
Minimax Victories	100
Dumb Agent Victories	0
Minimax Win Rate	100.0%
Dumb Agent Win Rate	0.0%
Average Game Length	11.64 moves
Starting Stone Range (n_0)	15 – 50



Analysis

- Deterministic Perfection:** Minimax achieved a **100.0% win rate**. Since the Halving Game exhibits complete observability without stochastic elements during play, Minimax successfully identifies forced winning sequences and fully exploits the predictable behavior of the Dumb Agent.
- Accelerated Convergence:** The overall average game length of **11.64 moves** (from an initial $n_0 \in [15, 50]$) demonstrates Minimax’s strategic utilization of the halving action ($\lfloor \frac{n}{2} \rfloor$) to shorten paths toward terminal winning states, rather than relying on sequential decrements.